



UTM

UNIVERSITI TEKNOLOGI MALAYSIA

SECR2043 OPERATING SYSTEM

SECTION 02

SEMESTER II, SESSION 2024/2025

Group Project

Title : Synchronization using Peterson Algorithm

Lecturer Name : Dr. Muhammad Zafran Bin Muhammad Zaly Shah

Video Link:

<https://drive.google.com/drive/folders/1as81sKcUVAvYoDlbPtZzTwTJnAmuahNm>

Submission Date : 6/7/2025

Group: Pacman

Name	Matric Number
LING LEE HOM	A23CS0105
PRAVINRAJ A/L SIVABATHI	A23CS0171
JUSTIN FAUZADANI AZKA	A23CS4012
DHESHIEGHAN A/L SARAVANA MOORTHY	A23CS0072
HARRESH A/L UTHAYAKUMAR	A23CS0226

Table of Contents

Abstract	
1.0 Introduction	1
2.0 Problem Statement	1
3.0 Related Works	2
4.0 Methodology	
4.1 Project Management Flowchart	3
4.2 Hardware and Software Requirements	4
4.3 Task Distribution	4
5.0 Design & Implementation	
5.1 System Flowchart	5
5.2 Code Implementation	6
6.0 Results & Discussion	
6.1 Interface and Execution	10
6.2 Output Analysis	10
6.3 Discussion	11
7.0 Future Work & Conclusion	
7.1 Future Work	12
7.2 Conclusion	12
8.0 Acknowledgement	13

Synchronization using Peterson Algorithm

Abstract

This project aims to provide a clear and intuitive understanding of process synchronization using Peterson's algorithm. Through visualization, we will demonstrate how Peterson's algorithm solves the critical section problem for two processes, ensuring mutual exclusion and progress. We will illustrate the algorithm's steps, including the use of flag and turn variables, within a relatable real-world scenario. This project will contribute to a deeper comprehension of fundamental operating system concepts related to concurrency and synchronization.

1.0 Introduction

In concurrent programming, multiple processes often need to access shared resources. Without proper coordination, this can lead to race conditions and data inconsistency. Synchronization mechanisms are crucial for managing access to shared resources and ensuring the correct execution of concurrent processes. Peterson's algorithm is a classic software-based solution to the critical section problem for two processes. This algorithm guarantees mutual exclusion in which only one process can be in its critical section at any time, if no process is in its critical section, a process wishing to enter will eventually do so, and bounded waiting a process will not wait indefinitely to enter its critical section. This project will explore the intricacies of Peterson's algorithm through a visual and explanatory presentation, making it accessible and understandable through a real-world analogy.

2.0 Problem Statement

The challenge of ensuring safe and efficient access to shared resources by multiple concurrent processes is a fundamental problem in operating systems. Without effective synchronization mechanisms, critical sections of code that access shared data can lead to inconsistent and incorrect results. Peterson's algorithm offers a software-based solution for synchronizing two processes, but its underlying logic and the way it achieves mutual exclusion, progress, and bounded waiting can be challenging to grasp without a clear and visual explanation. This project addresses the need for a readily understandable demonstration of Peterson's algorithm, illustrating its mechanism and benefits in preventing race conditions within a practical context.

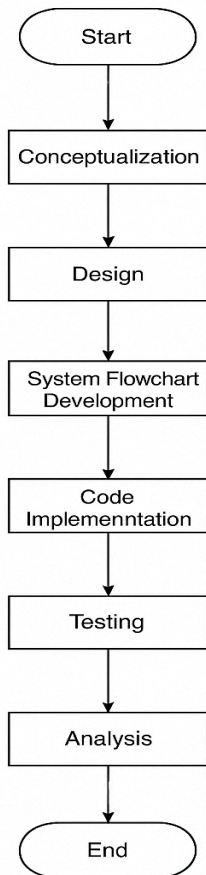
3.0 Related Works

The critical section problem and the need for process synchronization have been extensively studied in operating systems. Various hardware and software-based solutions have been proposed. Hardware solutions often rely on special machine instructions like Test-and-Set or Compare-and-Swap. Software-based solutions, such as Dekker's algorithm and Peterson's algorithm, aim to achieve synchronization using only basic memory operations. Peterson's algorithm, developed by Gary L. Peterson in 1981, is a significant early software-based solution known for its simplicity and elegance in solving the two-process critical section problem. While subsequent and more generalized synchronization primitives like semaphores and monitors are widely used in modern operating systems, understanding Peterson's algorithm provides a valuable foundation for comprehending the fundamental principles of mutual exclusion and concurrency control. This project builds upon the existing knowledge of Peterson's algorithm by focusing on creating a clear and engaging visual explanation to enhance learning and understanding.

4.0 Methodology

4.1 Project Management Flowchart

The Project Management Flowchart visually outlines the sequence of activities and phases involved in the project. It includes steps such as conceptualization, design, implementation, testing, and analysis.



Steps in the Flowchart:

1. **Conceptualization:** Define the objectives of the project, which is to demonstrate and explain the concept of process synchronization using Peterson's algorithm.
2. **Design:** Develop a clear model that outlines the synchronization mechanism using shared variables (**flag** and **turn**) to represent the logic behind Peterson's algorithm.
3. **System Flowchart Development:** Create a flowchart that visually explains how two processes interact, request access to the critical section, and follow the rules of mutual exclusion.
4. **Code Implementation:** Implement the simulation using C++. Use the `std::thread` library to simulate two concurrent threads representing processes, with **flag** and **turn** as shared variables. The critical section behavior is visualized through printed messages to the console.
5. **Testing:** Test the simulation by executing multiple runs, verifying that the output consistently satisfies mutual exclusion, progress, and bounded waiting. Use printed logs to trace the execution order and detect any synchronization issues.

6. **Analysis:** Analyze the printed output to confirm that Peterson's algorithm works correctly in all test cases. Reflect on its limitations and discuss how the results align with theoretical expectations.

4.2 Hardware and Software Requirements

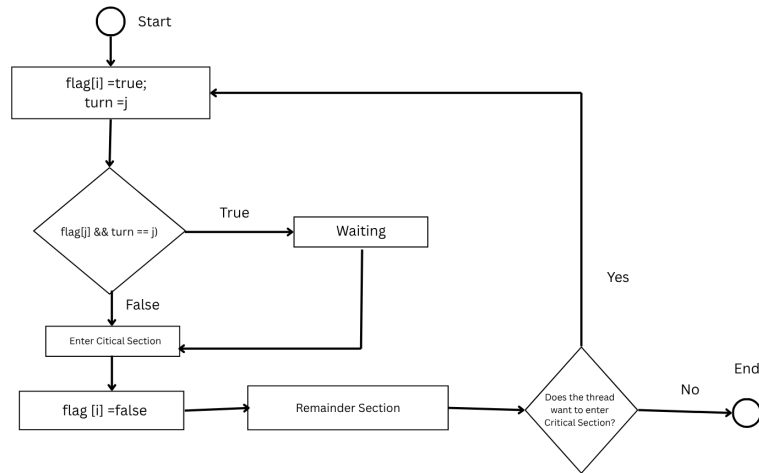
- **Software:**
C++ programming language (C++11 or higher). Uses standard C++ libraries with threading (`<thread>`), and output is visualized through `std::cout`.
- **Hardware:** Any standard laptop or desktop computer with a C++ compiler (such as g++, clang, or MSVC) capable of running multithreaded console-based applications.

4.3 Task Distribution

- **Literature Review:** Pravin
- **Algorithm Design and Flowchart:** Ling Lee Hom
- **Simulation Implementation and Testing:** Harresh & Justin
- **Visualization and Analysis:** Dheshieghan Saravana Moorthy
- **Report Writing and Presentation:** Everyone

5.0 Design & Implementation

5.1 System Flowchart



1. The thread sets `flag[i] = true` to indicate its intention to enter the critical section, and gives priority to the other thread by setting `turn = j`.
2. Determine if others wants to enter and it's their turn through while loop (`flag [j] && turn==j`)
 - 2.1. If true, it will go to the waiting loop,
 - 2.2. If false, it will directly enter to the critical section
3. After finishing the critical section, it sets `flag[i] = false` to allow the other thread to enter.
4. The thread enters the Remainder Section to perform other non-critical operations.
5. Does the thread want to enter the critical section?
 - 5.1. If yes , it needs to repeat the first step again.
 - 5.2. If not, the process will be end

5.2 Code Implementation

Initial Setup

```
1  #include <iostream>
2  #include <thread>
3  #include <atomic>
4  #include <chrono>
5
6  using namespace std;
7
8  // Shared variables
9  atomic<bool> flag[2];
10 atomic<int> turn;
11
12 // Shared resource
13 int sharedPrinterCounter = 0;
```

Import the required library (iostream, thread, atomic, chrono), then declare the global variable, which is **flag**, **turn** and **sharedCounter**. The **flag** array is used to indicate if a thread is ready to enter the critical section. **flag[i] = true** implies that process **i** is ready. The variable **turn** indicates whose turn it is to enter the critical section. The **sharedPrinterCounter** is used to count the number of the people(thread) that use the printing (Critical Section).

Declare function for usePrinter (Peterson Algorithm)

```
void usePrinter(int id) {
    int other = 1 - id;

    for (int i = 0; i < 5; i++) { // Each thread tries 5 times
        // Entry section
        flag[id] = true;
        turn = other;

        while (flag[other] && turn == other) {
            // Busy wait
        }
    }
}
```

The **other** variable is used to get the **id** of another person (thread). If the current thread is 1, then **other = 0**, vice versa. The **for** loop indicates that each person will go through the printing process (critical section) 5 times. The **while** loops indicate that if another person wants to enter as well, and it is now their turn, if true, the current person(thread) will enter this **while** loop and wait until

the other thread finishes their critical section. Then the current thread will go to the critical section.

Critical Section (Printing)

```
// Critical section
cout << (id == 0 ? "Alice" : "Bob") << " is using the printer...\n";
int temp = sharedPrinterCounter;
this_thread::sleep_for(chrono::milliseconds(100)); // Simulate some work
sharedPrinterCounter = temp + 1;
cout << (id == 0 ? "Alice" : "Bob") << " finished printing. Total prints: " << sharedPrinterCounter << "\n";
```

This code simulates the printing process (critical section). It will show which person already entered the printing process. Then it will wait for 100 ms to print the document. After that, the **sharedPrinterCounter** will increase by 1 for every printing process that it has performed. After that, messages related to the **sharedPrinterCounter** will be printed.

Remainder Section

```
// Exit section
flag[id] = false;

// Remainder section
this_thread::sleep_for(chrono::milliseconds(100));
}
}
```

After completing the printing (critical section) by the person (thread), the person's **flag** is set to false, indicating it has exited. It then enters the remainder section, where **this_thread::sleep_for()** is used to simulate processing by pausing for 100 milliseconds.

main function

```

int main() {

    flag[0] = false;
    flag[1] = false;
    thread alice(usePrinter, 0);

    thread bob(usePrinter, 1);

    alice.join();
    bob.join();

    cout << "All printing jobs completed.\n";
    return 0;
}

```

At first, it initializes the **flag** to become false. Then it declared the threads, which is **bob** and **alice**, to simulate two person intend to use the printer at the same time (exp: Click the ‘print’ button at the same time). Two of the threads run the **usePrinter()** function at the same time, **.join()** function is used to make sure that the process in the **main()** will only be continued after two of the threads have already finished. After that, a message will be printed to show the total amount that the threads have processed the printing (critical section).

Complete Code

```

1  #include <iostream>
2  #include <thread>
3  #include <atomic>
4  #include <chrono>
5
6  using namespace std;
7
8  // Shared variables
9  atomic<bool> flag[2];
10 atomic<int> turn;
11
12 // Shared resource
13 int sharedPrinterCounter = 0;
14
15 void usePrinter(int id) {
16     int other = 1 - id;
17
18     for (int i = 0; i < 5; i++) { // Each thread tries 5 times
19         // Entry section
20         flag[id] = true;
21         turn = other;
22
23         while (flag[other] && turn == other) {
24             // Busy wait
25         }
26
27         // Critical section
28         cout << (id == 0 ? "Alice" : "Bob") << " is using the printer...\n";
29         int temp = sharedPrinterCounter;
30         this_thread::sleep_for(chrono::milliseconds(100)); // Simulate some work
31     }
32 }

```

```

31     this_thread::sleep_for(chrono::milliseconds(100)); // Simulate some work
32     sharedPrinterCounter = temp + 1;
33     cout << (id == 0 ? "Alice" : "Bob") << " finished printing. Total prints: " << sharedPrinterCounter << "\n";
34
35     // Exit section
36     flag[id] = false;
37
38     // Remainder section
39     this_thread::sleep_for(chrono::milliseconds(100));
40 }
41 }
42
43 int main() {
44     flag[0] = false;
45     flag[1] = false;
46     thread alice(usePrinter, 0);
47
48     thread bob(usePrinter, 1);
49
50     alice.join();
51     bob.join();
52
53     cout << "All printing jobs completed.\n";
54     return 0;
55 }

```

Output

```

Alice is using the printer...
Alice finished printing. Total prints: 1
Bob is using the printer...
Bob finished printing. Total prints: 2
Alice is using the printer...
Alice finished printing. Total prints: 3
Bob is using the printer...
Bob finished printing. Total prints: 4
Alice is using the printer...
Alice finished printing. Total prints: 5
Bob is using the printer...
Bob finished printing. Total prints: 6
Alice is using the printer...
Alice finished printing. Total prints: 7
Bob is using the printer...
Bob finished printing. Total prints: 8
Alice is using the printer...
Alice finished printing. Total prints: 9
Bob is using the printer...
Bob finished printing. Total prints: 10
All printing jobs completed.

```

6.0 Result & Discussion

6.1 Interface and Execution

The implementation of the Peterson Algorithm was carried out using C++ with multithreading support via the `<thread>` and `<atomic>` libraries. The interface of the program is command-line based and displays messages that indicate when a thread is attempting to enter, currently inside, or has exited the critical section. Each thread runs the synchronization code five times to effectively demonstrate mutual exclusion.

The execution follows these steps:

- Each thread sets its intention to enter the critical section (`flag[i] = true`) and assigns the turn to the other thread (`turn = j`).
- If the other thread also wants to enter and it's their turn, the current thread waits (spins) until the other finishes.
- Once inside the critical section, the thread prints a message and increments a shared counter.
- After leaving the critical section, it resets its flag and enters a simulated remainder section using `sleep_for()`.

This execution sequence was repeated several times to ensure consistency and demonstrate synchronization without race conditions.

6.2 Output Analysis

The output from the program clearly shows that:

- No two threads are ever inside the critical section at the same time.
- Each thread enters the critical section exactly five times, resulting in a total of 10 increments in the shared counter.
- The messages printed to the terminal confirm the correct sequence of events: intent to enter, entry, processing in the critical section, and exit.
- The delay introduced in the critical and remainder sections (using `sleep_for(100ms)`) ensures that thread interleaving can occur, which validates the robustness of the synchronization.

6.3 Discussion

The results confirm that Peterson's Algorithm successfully enforces mutual exclusion, progress, and bounded waiting, the three essential conditions of the critical section problem:

- Mutual Exclusion: The shared counter is never updated by more than one thread at a time. No overlap in critical section entry was observed.
- Progress: If only one thread wants to enter the critical section, it is allowed to proceed without unnecessary delay.
- Bounded Waiting: Each thread gets fair access to the critical section without indefinite postponement.

Although Peterson's algorithm is limited to two processes and primarily used for educational purposes due to modern hardware optimizations, our implementation effectively illustrates the core synchronization challenge in a simple and clear way.

Furthermore, the simulation demonstrated how such synchronization logic can be critical in real-world systems where concurrent access to shared resources (like in Pacman's game states or high-score management) must be managed carefully.

7.0 Future Work & Conclusion

7.1 Future Work

While this project successfully demonstrated the effectiveness of Peterson's algorithm for two-process synchronization, several extensions and improvements can be explored in future work:

- **Integration with Graphical Visualization Tools:** Develop a more interactive, graphical user interface to visually represent the entry and exit of processes in the critical section. This would make the learning experience more intuitive and engaging for new learners.
- **Support for Multiple Processes:** Extend the implementation to support more than two processes by exploring generalizations such as the n-process version of Peterson's algorithm.
- **Performance Analysis on Modern Multicore Systems:** Evaluate Peterson's algorithm performance on modern multicore processors to highlight practical limitations and compare it with hardware-based synchronization primitives.

7.2 Conclusion

This project effectively demonstrated the core concepts of process synchronization using Peterson's algorithm. By implementing and analyzing the algorithm in a simulated two-thread environment, we confirmed that mutual exclusion, progress, and bounded waiting were successfully enforced. Hence, our results clearly showed that only one process entered the critical section at a time and all processes received fair access without indefinite delay.

Through this project, we gained a deeper understanding of concurrency issues and the importance of synchronization mechanisms in operating systems. Although Peterson's algorithm

is mainly of educational value today, it provides a strong foundation for understanding more advanced synchronization techniques. Future enhancements, including support for multiple processes and visualizations, can further strengthen its educational impact and practical insights.

8.0 Acknowledgement

We would like to express our sincere gratitude to our lecturer, Dr. Muhammad Zafran, for his continuous support, valuable guidance, and insightful feedback throughout this project. His expertise and clear explanations greatly enhanced our understanding of synchronization concepts and the implementation of Peterson's algorithm. We are truly grateful for his dedication and encouragement, which have been instrumental in helping us successfully complete this project.